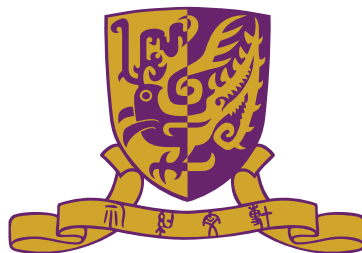


DDA5001 Machine Learning

Unsupervised Learning: Dimensionality Reduction

Xiao Li

School of Data Science
The Chinese University of Hong Kong, Shenzhen



Dimensionality Reduction

Principal Component Analysis (PCA)

Apply PCA to Real Image Dataset

Dimensionality Reduction

- Observe samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, without labels.

→ unsupervised learning.

Dimensionality reduction: Find a closest point to $\mathbf{x}_i$ in a lower dimensional space, i.e.,

$$\mathbb{R}^d \ni \mathbf{x}_i \rightarrow \tilde{\mathbf{x}}_i \in \mathbb{R}^k,$$

where $k \ll d$.

Dimensionality Reduction

- Observe samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, **without labels**.

Dimensionality reduction: Find a **closest** point to $\mathbf{x}_i$ in a **lower dimensional space**, i.e.,

$$\mathbb{R}^d \ni \mathbf{x}_i \rightarrow \tilde{\mathbf{x}}_i \in \mathbb{R}^k,$$

where $k \ll d$.

- Contrary to kernel methods in supervised learning.

Dimensionality Reduction

- ▶ Observe samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, **without labels**.

Dimensionality reduction: Find a **closest** point to $\mathbf{x}_i$ in a **lower dimensional space**, i.e.,

$$\mathbb{R}^d \ni \mathbf{x}_i \rightarrow \tilde{\mathbf{x}}_i \in \mathbb{R}^k,$$

where $k \ll d$.

- ▶ Contrary to kernel methods in supervised learning.

The motivation of dimensionality reduction:

- ▶ Reducing redundant information.
- ▶ Help algorithms to be more computationally efficient (in lower dimension).
- ▶ **Preventing overfitting**, especially when $n < d$ (data preprocessing for supervised learning).

Dimensionality Reduction

- Observe samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, **without labels**.

Dimensionality reduction: Find a **closest** point to $\mathbf{x}_i$ in a **lower dimensional space**, i.e.,

$$\mathbb{R}^d \ni \mathbf{x}_i \rightarrow \tilde{\mathbf{x}}_i \in \mathbb{R}^k,$$

where $k \ll d$.

- Contrary to kernel methods in supervised learning.

The motivation of dimensionality reduction:

- Reducing redundant information.
- Help algorithms to be more computationally efficient (in lower dimension).
- Preventing overfitting, especially when $n < d$ (data preprocessing for supervised learning).

Dimensionality reduction is an important **unsupervised learning** technique. The main methods for dimensionality reduction are **feature selection** and **feature extraction**. We will focus on the later.

Dimensionality Reduction

Principal Component Analysis (PCA)

Apply PCA to Real Image Dataset

Principal Component Analysis

Principal component analysis (PCA): Find a low-dimensional approximation of high-dimensional data by minimizing the squared norms of distances.

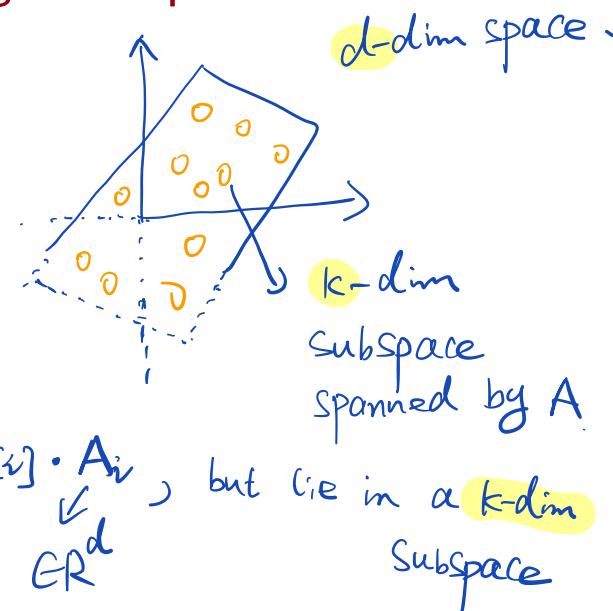
Principal Component Analysis

Principal component analysis (PCA): Find a **low-dimensional approximation** of high-dimensional data by **minimizing the squared norms of distances**.

PCA modeling of data:

$$\mathbb{R}^d \ni x \approx A\theta + \mu$$

$\tilde{x}$
↓
 $\mathbb{R}^d$

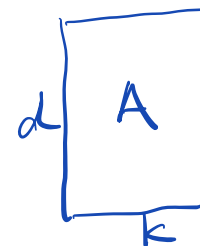


$$\tilde{x} = A\theta = \sum_{i=1}^k \theta[i] \cdot A_i$$

↓
 $\mathbb{R}^d$

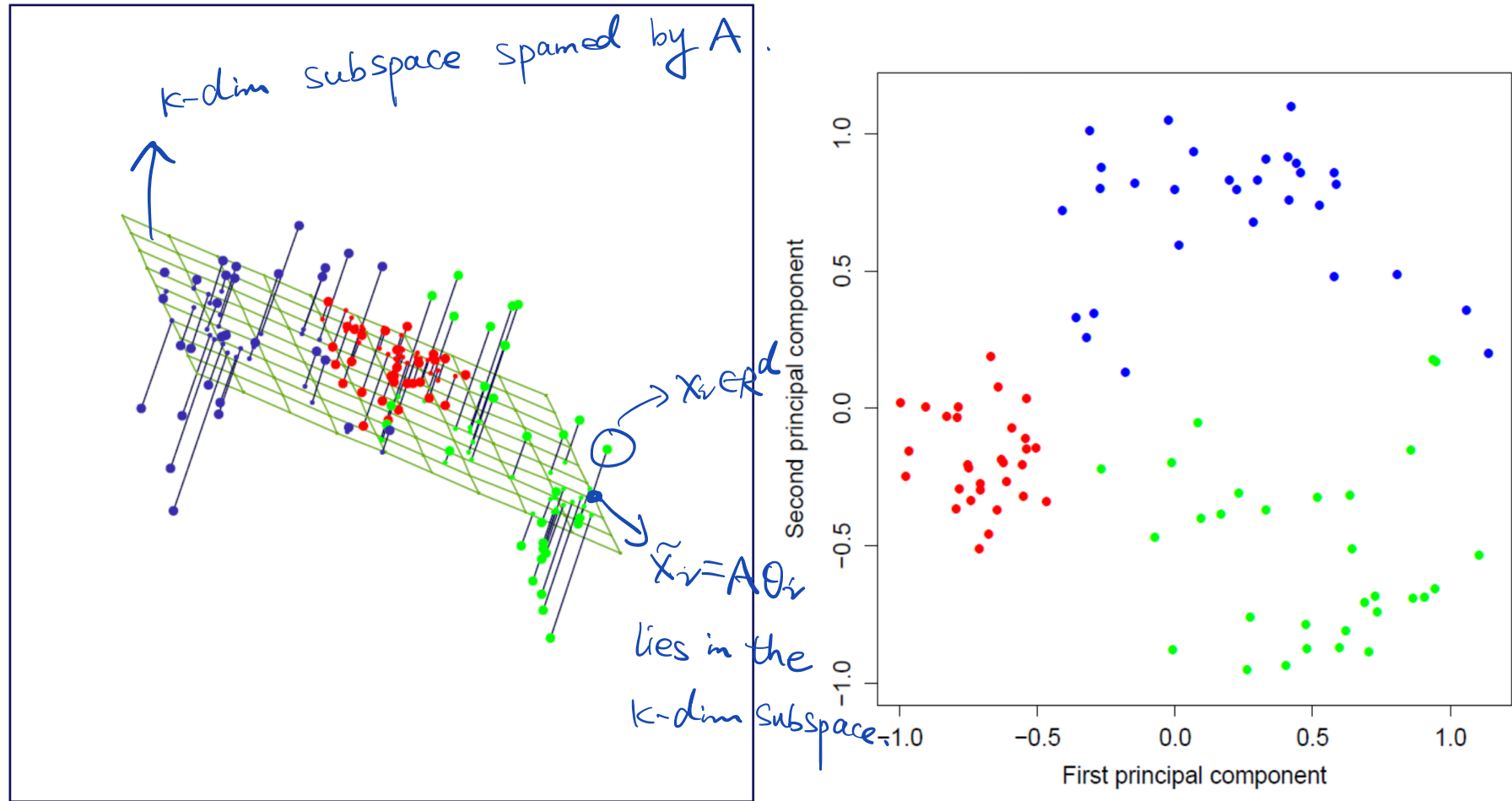
but lie in a **k-dim subspace**

- ▶ $x \in \mathbb{R}^d$ is the original sample.
- ▶ $A = [A_1, \dots, A_k] \in \mathbb{R}^{d \times k}$ with orthogonal columns, i.e., satisfying $A^T A = I_k$. Matrix A is often called **basis**. but $AA^T \neq I$, semi-orthogonal
- ▶ $\theta \in \mathbb{R}^k$ is the **principle component**.
- ▶ μ is the **mean** of the samples.



Interpretation: x (after removing the mean μ) can be approximated by a **k-dimensional** point $\tilde{x} = A\theta$.

Principal Component Analysis - Illustration



The PCA Learning Problem

PCA boils down to

$$\underset{\boldsymbol{\mu}, \{\boldsymbol{\theta}_i\}, \mathbf{A}^\top \mathbf{A} = \mathbf{I}_k}{\text{minimize}} \quad \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{A}\boldsymbol{\theta}_i - \boldsymbol{\mu}\|_2^2$$

- ▶ Only samples $\{\mathbf{x}_i\}$ are known. Others are unknowns.
- ▶ It is a nonconvex optimization problem.
- ▶ The hard part is to solve for $\mathbf{A}$.
- ▶ Given $\mathbf{A}$, finding $\boldsymbol{\mu}$ and $\{\boldsymbol{\theta}_i\}$ is easy.

Solve for θ_i

Given A, μ

$$\underset{\{\theta_i\}}{\text{minimize}} \sum_{i=1}^n \underbrace{\|x_i - A\theta_i - \mu\|_2^2}_{\text{minimize}}$$

Solution:

$$\theta_i = A^\top (x_i - \mu)$$

Why?

$$\begin{aligned} \theta_i &= A^\top (x_i - \mu) = \underbrace{(A^\top A)^{-1}}_{I_k} A^\top (x_i - \mu) \\ &= A^\top (x_i - \mu), \quad i=1, \dots, n \end{aligned}$$

Solve for θ_i

Given A, μ

$$\underset{\{\theta_i\}}{\text{minimize}} \sum_{i=1}^n \|x_i - A\theta_i - \mu\|_2^2.$$

Solution:

$$\theta_i = A^\top (x_i - \mu)$$

Why? It is just a standard least square problem with A being semi-orthogonal.

Solve for μ

Suppose given $\mathbf{A}$, setting $\theta_i = \mathbf{A}^\top (\mathbf{x}_i - \mu)$, we have

$$\underset{\mu}{\text{minimize}} \sum_{i=1}^n \|\mathbf{x}_i - \underbrace{\mathbf{A}\mathbf{A}^\top (\mathbf{x}_i - \mu)}_{\theta_i} - \mu\|_2^2.$$

It is equivalent to

$$\underset{\mu}{\text{minimize}} \sum_{i=1}^n \|(\mathbf{I} - \mathbf{A}\mathbf{A}^\top)(\mathbf{x}_i - \mu)\|_2^2.$$

$$\|a\|^2 = a^\top a$$

It is further equivalent to

$$\underset{\mu}{\text{minimize}} \sum_{i=1}^n (\mathbf{x}_i - \mu)^\top \underbrace{(\mathbf{I} - \mathbf{A}\mathbf{A}^\top)^\top (\mathbf{I} - \mathbf{A}\mathbf{A}^\top)}_{\mathbf{B}} (\mathbf{x}_i - \mu).$$

$$\text{Let } \mathbf{B} = (\mathbf{I} - \mathbf{A}\mathbf{A}^\top)^\top (\mathbf{I} - \mathbf{A}\mathbf{A}^\top) = \mathbf{I} - \mathbf{A}\mathbf{A}^\top.$$

Solve for μ

Take the gradient with respect to μ gives

$$\nabla_{\mu} = 2 \sum_{i=1}^n B(x_i - \mu) = 0$$

$\searrow$

$$= 2 \cdot B \left(\underbrace{\sum_{i=1}^n (x_i - \mu)}_{=0} \right) = 0$$

Set the gradient to zero yields **one** solution

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

$\downarrow$
Sample average

Solve for A

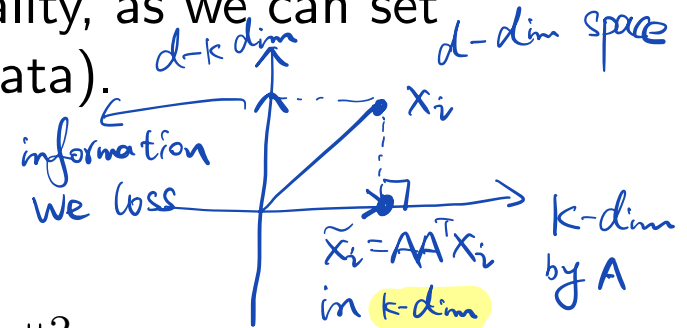
It remains to solve

$$\underset{A^T A = I}{\text{minimize}} \sum_{i=1}^n \|x_i - \overbrace{AA^T}^{\theta_i} (x_i - \mu) - \mu\|_2^2$$

- We can assume $\mu = 0$ without loss of generality, as we can set $x_i = x_i - \mu$ (removing the mean from the data).

The problem reduces to

$$\underset{A^T A = I}{\text{minimize}} \sum_{i=1}^n \|x_i - \overbrace{AA^T}^{\theta_i} x_i\|_2^2$$



Fact: AA^T is a projection matrix to the k -dim subspace spanned by A .

This is one form of PCA.

- **Interpretation:** Find the **closest** k -dimensional data point $AA^T x_i = \tilde{x}_i$ to x_i (projecting x_i onto the k -dimensional subspace spanned by the columns of A).

Derivation of the Second Equivalent PCA Form

Given the PCA formulation

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{minimize}} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{A}\mathbf{A}^\top \mathbf{x}_i\|_2^2.$$

Expanding the objective function yields

$$\begin{aligned} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{A}\mathbf{A}^\top \mathbf{x}_i\|_2^2 &= \sum_{i=1}^n (\mathbf{x}_i - \mathbf{A}\mathbf{A}^\top \mathbf{x}_i)^\top (\mathbf{x}_i - \mathbf{A}\mathbf{A}^\top \mathbf{x}_i) \\ &= \sum_{i=1}^n \left(\mathbf{x}_i^\top \mathbf{x}_i - 2\mathbf{x}_i^\top \mathbf{A}\mathbf{A}^\top \mathbf{x}_i + \underbrace{\mathbf{x}_i^\top \mathbf{A}\mathbf{A}^\top \mathbf{A}\mathbf{A}^\top \mathbf{x}_i}_{\mathbf{I}} \right) \\ &= \sum_{i=1}^n \mathbf{x}_i^\top \mathbf{x}_i - \mathbf{x}_i^\top \mathbf{A}\mathbf{A}^\top \mathbf{x}_i. \end{aligned}$$

We reduce to the **second form** of PCA

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \sum_{i=1}^n \mathbf{x}_i^\top \mathbf{A}\mathbf{A}^\top \mathbf{x}_i = \sum_{i=1}^n \underbrace{\|\mathbf{A}^\top \mathbf{x}_i\|_2^2}_{= \|\mathbf{A}\mathbf{A}^\top \mathbf{x}_i\|^2}.$$

Derivation of the Third Equivalent PCA Form

The second form is further equivalent to $\bullet \text{tr}(AB) = \text{tr}(BA)$

$$\begin{aligned}
 & \underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \sum_{i=1}^n \underbrace{\|\mathbf{A}^\top \mathbf{x}_i\|_2^2}_{\text{GR}} = \frac{\mathbf{x}_i^\top \mathbf{A} \mathbf{A}^\top \mathbf{x}_i}{\text{GR}} = \text{tr}(\underbrace{\mathbf{x}_i^\top \mathbf{A} \mathbf{A}^\top \mathbf{x}_i}_{= \text{tr}(\mathbf{A}^\top \mathbf{x}_i \mathbf{x}_i^\top \mathbf{A})}) \\
 & \iff \underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \sum_{i=1}^n \text{trace}(\mathbf{A}^\top \mathbf{x}_i \mathbf{x}_i^\top \mathbf{A}) \leftarrow \\
 & \iff \underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \text{trace} \left(\mathbf{A}^\top \left(\sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top \right) \mathbf{A} \right).
 \end{aligned}$$

Let

$$\mathbf{S} = \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top = \mathbf{X} \mathbf{X}^\top, \quad \text{where } \mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n] \in \mathbb{R}^{d \times n}$$

be the **empirical covariance matrix**. We have the following **third form** of PCA

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \text{trace}(\mathbf{A}^\top \mathbf{S} \mathbf{A}).$$

We use the third form to derive the solution for $\mathbf{A}$. We need to consult a matrix computation tool called **eigen decomposition**.

Eigenvalue Analysis

Eigenvalue problem: Given matrix S , find a vector u and a scalar λ such that

$$Su = \lambda u$$

- ▶ λ characterizes the behavior of S in u .
- ▶ u is called the **eigenvector**, while λ is called the **eigenvalue**.

Eigen decomposition for real PSD matrix

Suppose matrix $S \in \mathbb{R}^{d \times d}$ is **real**, **symmetric**, and **positive semidefinite (PSD)**, it always admits an eigen decomposition:

$$\Lambda = \begin{bmatrix} \lambda_1 & & 0 \\ & \ddots & \\ 0 & & \lambda_d \end{bmatrix}$$

$$S = U \Lambda U^T$$

$$U = [u_1 \cdots u_k \quad u_{k+1} \cdots u_d]$$

where $U \in \mathbb{R}^{d \times d}$ is an orthogonal matrix satisfying $U^T U = U U^T = \mathbf{I}$ containing eigenvectors and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d)$ with $\lambda_1 \geq \dots \geq \lambda_d \geq 0$ is a diagonal matrix containing the corresponding eigenvalues.

Return to The Solution of PCA

Consider the third form of PCA

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \quad \text{trace} \left(\mathbf{A}^\top \mathbf{S} \mathbf{A} \right),$$

where $\mathbf{S} = \mathbf{X} \mathbf{X}^\top$.

- ▶ Note that $\mathbf{S}$ is constructed from the data matrix $\mathbf{X}$, it is computable.
- ▶ Further, $\mathbf{S}$ is real and **must be PSD** (why?).

$$\mathbf{v}^\top \mathbf{S} \mathbf{v} = \mathbf{v}^\top \mathbf{X} \mathbf{X}^\top \mathbf{v}$$

$$= \|\mathbf{X}^\top \mathbf{v}\|_2^2 \geq 0.$$

Thus we apply eigen decomposition to $\mathbf{S}$ to obtain

$$\mathbf{S} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top.$$

The PCA problem becomes

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \quad \text{trace} \left(\mathbf{A}^\top \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top \mathbf{A} \right).$$

Return to The Solution of PCA Φ

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \text{ trace} \left(\mathbf{A}^\top \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top \mathbf{A} \right)$$

$$\Phi \Phi^\top = \mathbf{A}^\top \mathbf{U} \mathbf{U}^\top \mathbf{A} = \mathbf{A}^\top \mathbf{A} = \mathbf{I}$$

Let $\Phi = \mathbf{A}^\top \mathbf{U} \in \mathbb{R}^{k \times d}$, it is a semi-orthogonal matrix since $\Phi \Phi^\top = \mathbf{I}$

We can rewrite the optimization problem as

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \text{ trace} \left(\Phi \mathbf{\Lambda} \Phi^\top \right) = \underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \text{ trace} \left(\sum_{i=1}^d \lambda_i \phi_i \phi_i^\top \right)$$

$\Phi = [\phi_1, \dots, \phi_d]$

put the trace inside, we have

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \sum_{i=1}^d \lambda_i \phi_i^\top \phi_i = \sum_{i=1}^k \lambda_i$$

Fact: This optimization problem has upper bound $\leq \sum_{i=1}^k \lambda_i$. Hence, it attains its maximum when $\phi_i^\top \phi_i = 1, i = 1, \dots, k$ and $\phi_i^\top \phi_i = 0, i = k + 1, \dots, d$. This is achieved by

$\phi_i = \mathbf{A}^\top \mathbf{u}_i = \begin{cases} \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, i=1, \dots, k \\ \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}, i=k+1, \dots, d. \end{cases}$

$$\mathbf{A} = [\mathbf{u}_1, \dots, \mathbf{u}_k]$$

$\mathbf{A}^\top = \begin{bmatrix} \mathbf{u}_1^\top \\ \vdots \\ \mathbf{u}_k^\top \end{bmatrix}$

i.e., the first k eigenvectors.

Process of Computing The PCA

Given samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, **without labels**.

- ▶ Remove the mean

$$\mathbf{x}_i = \mathbf{x}_i - \boldsymbol{\mu},$$

where $\boldsymbol{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$.

- ▶ Form the empirical covariance matrix from data

$$\mathbf{S} = \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top = \mathbf{X} \mathbf{X}^\top.$$

- ▶ Compute the eigen decomposition of $\mathbf{S}$

$$\mathbf{S} = \mathbf{U} \boldsymbol{\Lambda} \mathbf{U}^\top$$

and the PCA solution is given by

$$\mathbf{A} = [\mathbf{u}_1, \dots, \mathbf{u}_k].$$

- ▶ Compute principle component and low-dimensional sample

$$\boldsymbol{\theta}_i = \mathbf{A}^\top \mathbf{x}_i, \quad \tilde{\mathbf{x}}_i = \mathbf{A} \boldsymbol{\theta}_i = \mathbf{A} \mathbf{A}^\top \mathbf{x}_i.$$

Connection to Singular Value Decomposition

Singular value decomposition (SVD)

Given **any** real matrix $\mathbf{X} \in \mathbb{R}^{d \times n}$, there exists a 3-tuple $(\mathbf{U}, \mathbf{\Sigma}, \mathbf{V}) \in \mathbb{R}^{d \times d} \times \mathbb{R}^{d \times n} \times \mathbb{R}^{n \times n}$ such that

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\top},$$

where $\mathbf{U}$ and $\mathbf{V}$ are orthogonal and $\mathbf{\Sigma}$ takes the form

$$\Sigma(i, j) = \begin{cases} \sigma_i, & i = j \\ 0, & i \neq j \end{cases}$$

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0, \quad p = \min(d, n).$$

- ▶ σ_i are called **singular value**.
- ▶ $\mathbf{u}_i$ and $\mathbf{v}_i$ are called **left and right singular vectors**.

PCA via SVD

Recall $S = \mathbf{X}\mathbf{X}^\top$, the eigen decomposition is $S = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$, and the solution to the PCA problem is

$$\mathbf{A} = [\mathbf{u}_1, \dots, \mathbf{u}_k]$$

We can instead compute the SVD of matrix $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ and we also have

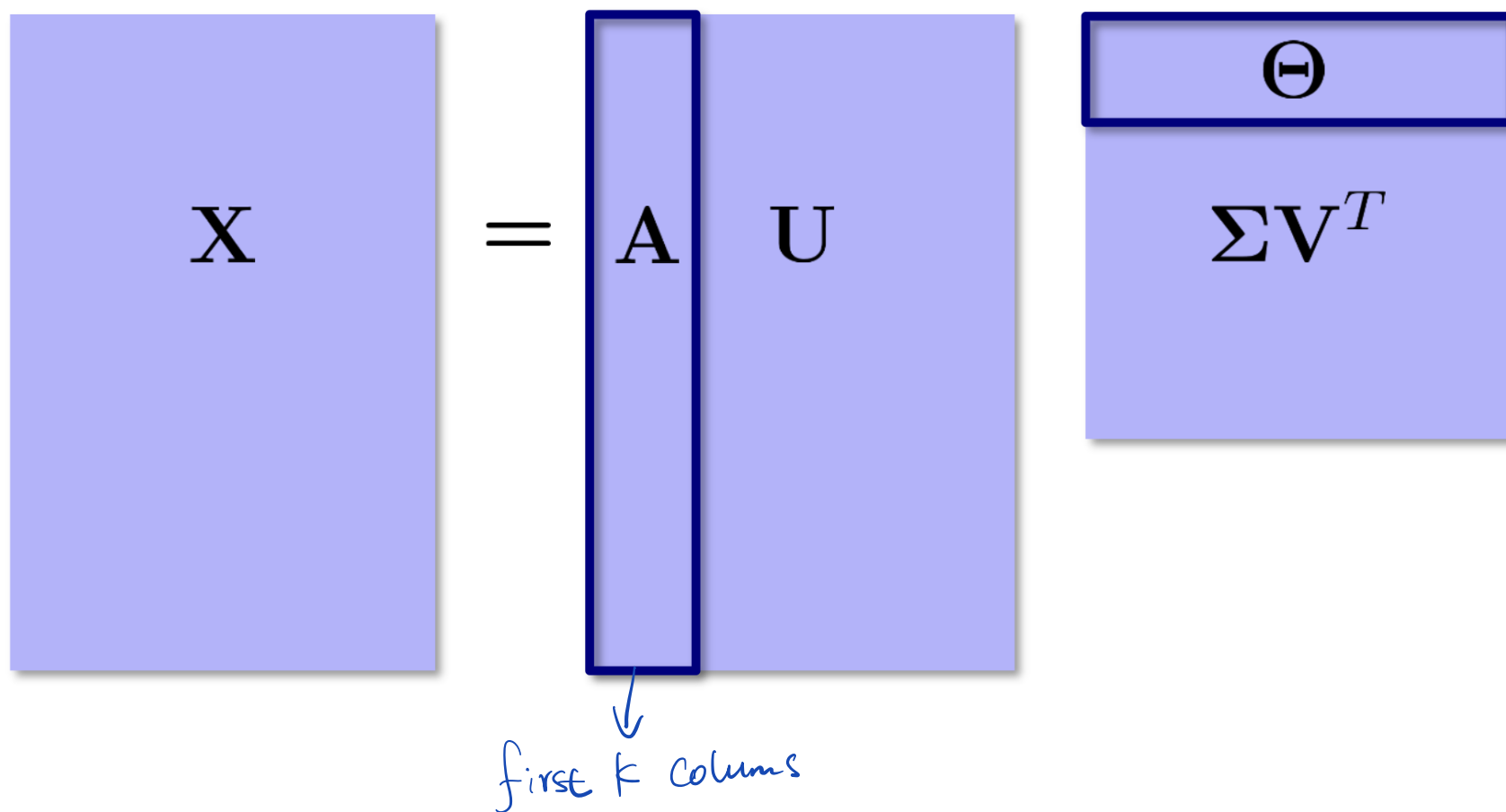
$$\mathbf{A} = [\mathbf{u}_1, \dots, \mathbf{u}_k].$$

$$\begin{aligned}\mathbf{X}\mathbf{X}^\top &= \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top\mathbf{V}\mathbf{\Sigma}\mathbf{U}^\top \\ &= \mathbf{U}\mathbf{\Sigma}^2\mathbf{U}^\top\end{aligned}$$

- ▶ Computing PCA via SVD can be more favorable, as it can be more **numerically reliable** than eigen decomposition.
- ▶ It might also **save computation time** as we do not need to compute $S = \mathbf{X}\mathbf{X}^\top$, which can be expensive when $\mathbf{X}$ is a large matrix.

PCA via SVD: Illustration

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$



PCA From the Matrix Factorization Perspective

Recall the (first form) PCA (we assume without loss of generality that $\mu = 0$)

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}, \{\boldsymbol{\theta}_i\}}{\text{minimize}} \quad \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{A}\boldsymbol{\theta}_i\|_2^2.$$

It can be written in a matrix form:

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}, \boldsymbol{\Theta}}{\text{minimize}} \quad \|\mathbf{X} - \mathbf{A}\boldsymbol{\Theta}\|_F^2$$

where $\mathbf{A} \in \mathbb{R}^{d \times k}$ and $\boldsymbol{\Theta} \in \mathbb{R}^{k \times n}$, and $\|\cdot\|_F$ is the **Frobenius norm**.

- ▶ The above problem is also called **low-rank matrix factorization**.
- ▶ **Interpretation:** Factorize $\mathbf{X}$ into two factors' multiplication, where the latter is a **low-rank matrix**.

PCA From the Matrix Factorization Perspective

Low-rank matrix factorization

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}, \mathbf{\Theta}}{\text{minimize}} \quad \|\mathbf{X} - \mathbf{A}\mathbf{\Theta}\|_F^2$$

- ▶ Calculate the SVD of $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$.

Solution from PCA

- ▶ One optimal solution to the above (low-rank) matrix factorization problem is given by

$$\mathbf{A} = [\mathbf{u}_1, \dots, \mathbf{u}_k], \quad \mathbf{\Theta} = [\sigma_1 \mathbf{v}_1, \dots, \sigma_k \mathbf{v}_k]^\top$$

- ▶ It has infinitely many equivalent optimal solutions.

It is a **closed-form solution** to a nonconvex optimization problem.

More General Matrix Factorization

We can also remove the orthogonal constraint on $\mathbf{A}$ to allow more flexibility

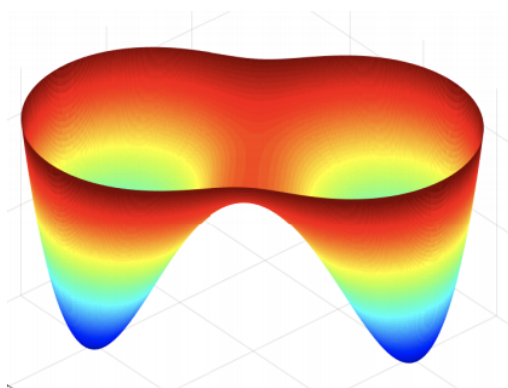
$$\underset{\mathbf{A} \in \mathbb{R}^{d \times k}, \mathbf{\Theta} \in \mathbb{R}^{k \times n}}{\text{minimize}} \quad \|\mathbf{X} - \mathbf{A}\mathbf{\Theta}\|_F^2.$$

One optimal solution to the above (low-rank) matrix factorization problem is given by

$$\mathbf{A} = [\sqrt{\sigma_1} \mathbf{u}_1, \dots, \sqrt{\sigma_k} \mathbf{u}_k], \quad \mathbf{\Theta} = [\sqrt{\sigma_1} \mathbf{v}_1, \dots, \sqrt{\sigma_k} \mathbf{v}_k]^\top,$$

and it has infinitely many equivalent optimal solution.

- A nonconvex optimization problem



- Fortunately, closed-form solution exists.

LoRA: Low-rank Adaptation

In the **post-training** stage of large models (like large language models), we often need to learn an incremental to the learned model to incorporate new knowledge. That is

$$\underset{\Delta\Theta \in \mathbb{R}^{m \times n}}{\text{minimize}} \quad \mathcal{L}(\hat{\Theta} + \Delta\Theta)$$

Low-rank Adaptation (LoRA) uses the simple idea of PCA. It does not aim to learn the full $\Delta\Theta$, it instead learns a **PCA decomposition** of $\Delta\Theta$.

$$\underset{\mathbf{A} \in \mathbb{R}^{m \times r}, \mathbf{B} \in \mathbb{R}^{r \times n}}{\text{minimize}} \quad \mathcal{L}(\hat{\Theta} + \mathbf{AB})$$

where $r \ll \min\{m, n\}$.

- ▶ LoRA approach can **save computation memory**.
- ▶ It is now widely utilized in large language models.
- ▶ We will use LoRA in our final project.

Dimensionality Reduction

Principal Component Analysis (PCA)

Apply PCA to Real Image Dataset

The ORL Database of Faces



- ▶ 40 persons.
- ▶ Each has 10 distinct face images.
- ▶ Each image is of size 92×112 .
- ▶ The images were taken at different times, varying the lighting, facial expressions (open / closed eyes, smiling / not smiling) and facial details (glasses / no glasses).

Form the Data Matrix



92x112

vectorize $\longrightarrow$

$$\mathbf{x}_i = \begin{bmatrix} \mathbf{x}_i[1] \\ \vdots \\ \mathbf{x}_i[92 \times 112] \end{bmatrix} \in \mathbb{R}^{10304}$$

- Do the same thing for all the face images, we get

$$\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n] \in \mathbb{R}^{d \times n},$$

where $d = 10304$, $n = 10 \times 40 = 400$.

Apply PCA to $\mathbf{X}$ with $k = 40$

Perform PCA, i.e., solving

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}, \Theta}{\text{minimize}} \quad \|\mathbf{X} - \mathbf{A}\Theta\|_F^2$$

We get the **extracted features** (i.e., $\mathbf{A} \in \mathbb{R}^{d \times k}$)



where we resize each column of $\mathbf{A}$ (of dimension $d = 10304$) to a 92×112 image and show it.

Application

$$\mathbf{x}_i = \mathbf{A}\boldsymbol{\theta}_i$$


- ▶ Each face image $\mathbf{x}_i$ can be interpreted as a linear combination of the columns in $\mathbf{A}$.
- ▶ The importance of each feature is implied in $\boldsymbol{\theta}_i$.
- ▶ $\mathbf{x}_i \in \mathbb{R}^d$ ($d = 10304$ here) has been nicely represented by a k -dimensional vector $\mathbf{A}\boldsymbol{\theta}_i$ ($k = 40$ here) — Dimensionality reduction.

⇒ Next lecture: Unsupervised learning: Clustering and k-means.